DNSSEC

From “What is a zone?” to a complete chain of trust

Goal: understand signatures, keys, delegation, and validation—not memorize record names.

Roadmap: build DNSSEC one layer at a time

PLAN

We start with ordinary DNS, then add authenticity without changing the lookup model.

1. DNS zones
Who is authoritative for what?

2. RRsets
What exactly gets signed?

3. RRSIG
How is an RRset authenticated?

4. ZSK & KSK
Why are there two key roles?

5. DNSKEY & DS
How do keys appear in DNS?

6. Chain of trust
How does a resolver validate end-to-end?

Why DNSSEC exists

MOTIVATION

DNS normally tells you an answer. DNSSEC lets a validating resolver check that the answer is authentic.

User asks:
www.example.com?

DNS reply:
203.0.113.10

Question:
Can I trust it?

The problem

- Classic DNS has no built-in cryptographic proof of origin.
- A forged DNS reply may redirect a client.
- DNSSEC adds digital signatures to DNS data.

What DNSSEC provides

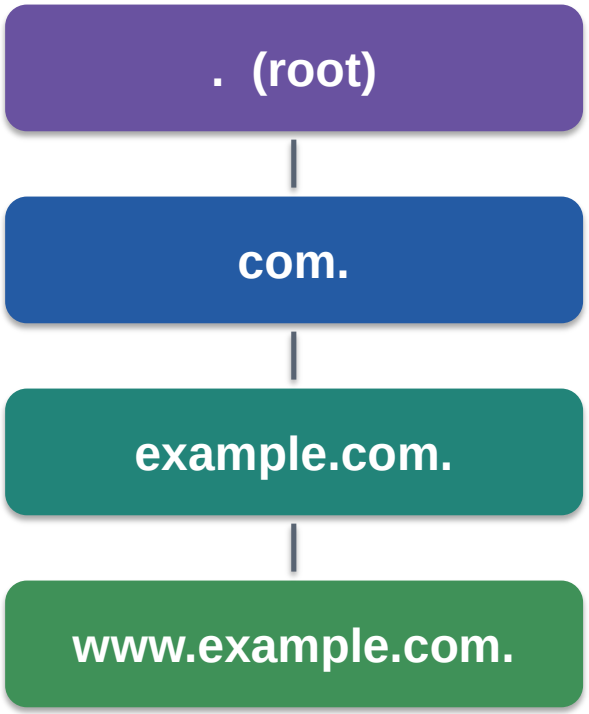
- Data origin authentication
- Data integrity
- Authenticated denial of existence

DNSSEC does NOT encrypt DNS and does NOT hide which name you query.

First: remember the DNS hierarchy

A domain name is read from the most specific label toward the root.

Each level can delegate responsibility to another set of authoritative name servers.



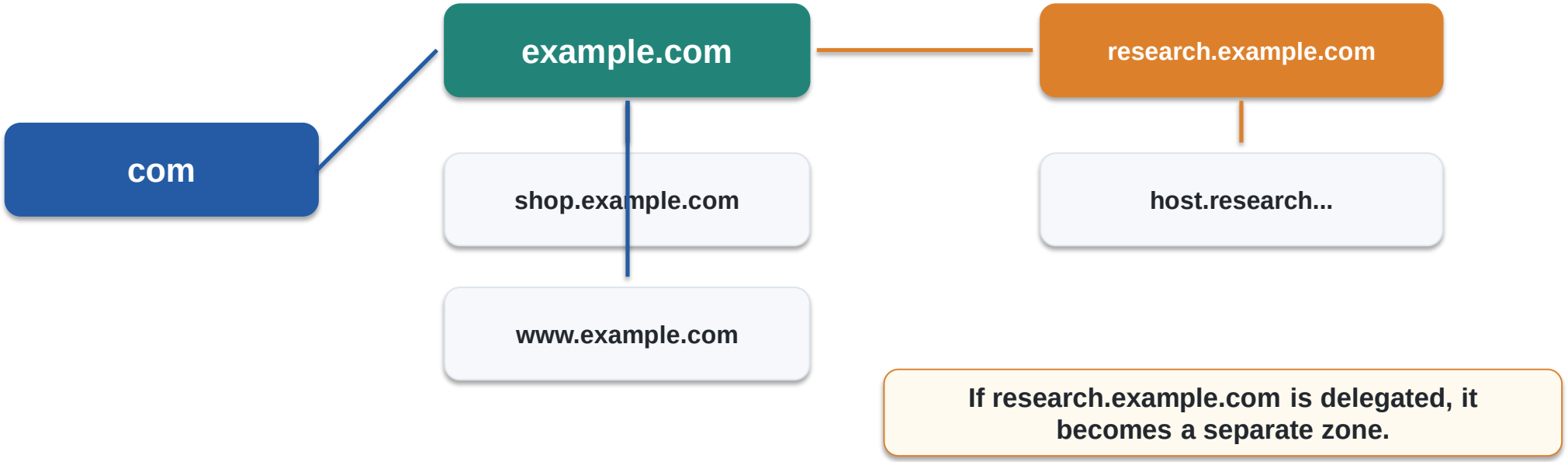
DNSSEC follows this hierarchy: trust is passed from parent to child.

What exactly is a DNS zone?

ZONE

This concept is essential: DNSSEC signs a zone’s data.

A zone is the portion of the DNS namespace for which a particular administrative authority publishes authoritative DNS data.



Zone ≠ domain

ZONE

A domain is a naming subtree. A zone is an administrative slice of that subtree.

DOMAIN: example.com

Includes every name below example.com conceptually:

www.example.com
mail.example.com
research.example.com
host.research.example.com

ZONE: example.com

Contains authoritative data managed by this zone.

If research.example.com is delegated, its internal records are no longer in the example.com zone.

DNSSEC keys and signatures are managed per zone.

What is inside a zone?

ZONE

Think of a zone as a database of DNS resource records.

```
example.com.          3600  SOA   ns1.example.com. ...
example.com.          3600  NS    ns1.example.com.
www.example.com.      300   A     203.0.113.10
mail.example.com.     300   A     203.0.113.20
example.com.          300   MX    10 mail.example.com.
```

DNSSEC will add more record types to this same zone: DNSKEY, RRSIG, NSEC/NSEC3.

Before RRSIG: understand an RRset

RRSET

DNSSEC signs an RRset—not an individual record in isolation.

```
www.example.com.    300    A    203.0.113.10
www.example.com.    300    A    203.0.113.11
www.example.com.    300    A    203.0.113.12
```

Same owner name
+
Same record type
=
ONE RRset

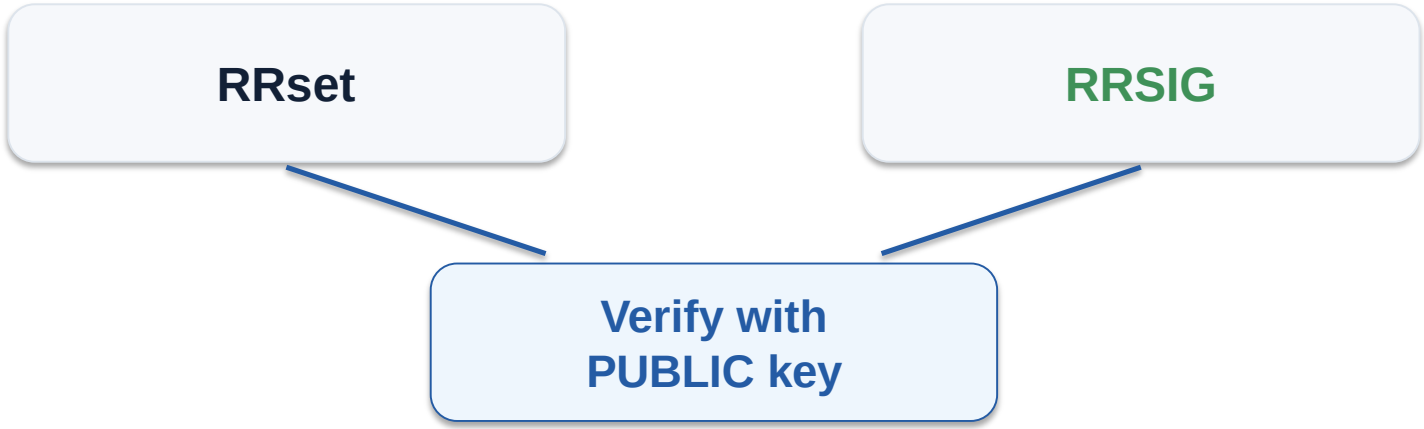
The three A records above form the “www.example.com A RRset”.

One signature can authenticate the complete set. DNS answers may contain multiple records of the same type.

Now add a digital signature

RRSIG

The zone operator signs an RRset with a private key. Validators check it with the matching public key.



If the data changes, the signature verification fails.

What is an RRSIG record?

RRSIG

RRSIG is the DNS record that carries a digital signature over one RRset.

```
www.example.com.  300  RRSIG  A 13 3 300 202609... 202608... 12345 example.com. <signature>
```

Important fields

- Type covered: which RRset was signed (e.g., A)
- Algorithm: cryptographic algorithm
- Inception / expiration: signature validity window
- Key tag: identifies the DNSKEY used
- Signer name: the signing zone

Avoid these confusions

- RRSIG is not the key.
- RRSIG is not a certificate.
- RRSIG does not encrypt the A record.
- A zone has many RRSIG records—typically one per signed RRset.

RRSIG = “Here is the cryptographic proof for this RRset.”

But where does the public key come from?

DNSKEY

A signature is useful only if the validator can obtain the corresponding public key.

DNSSEC publishes the zone's public keys inside DNS itself, using DNSKEY records.

```
example.com.        3600  DNSKEY  256 3 13 <public-key-material>
```

Private key: kept secret by the zone operator
Public key: published as a DNSKEY record

DNSKEY is a record format—not a third kind of key

DNSKEY

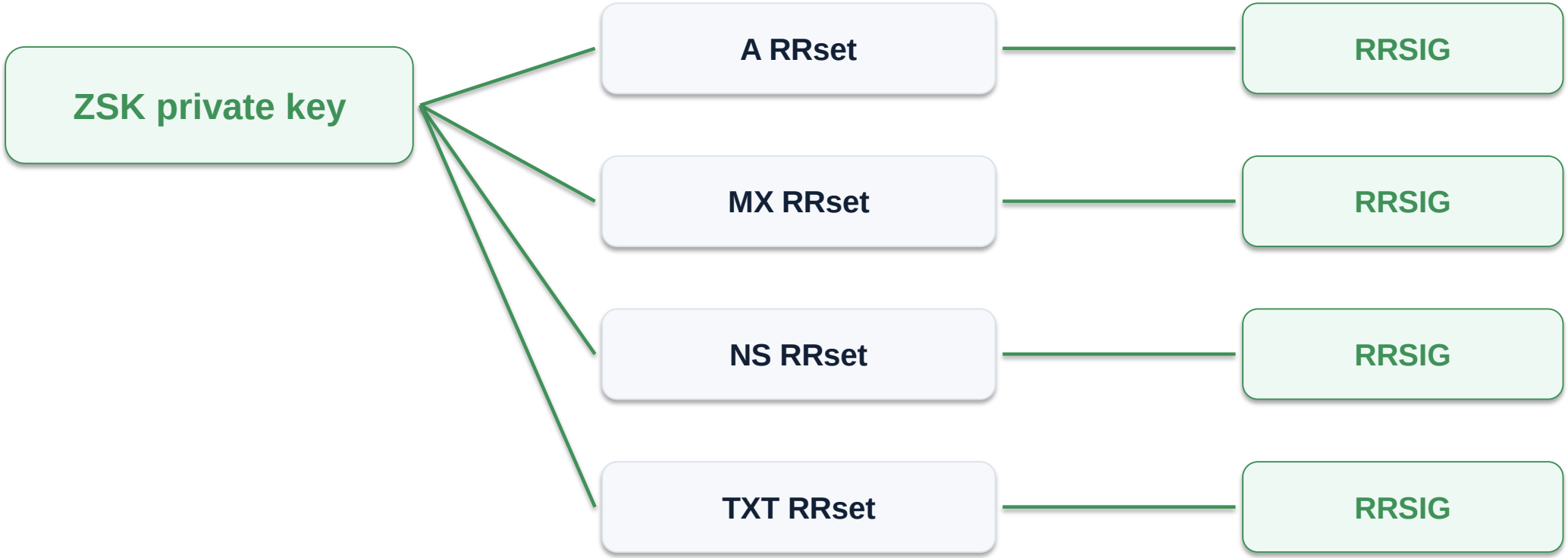
This is one of the most common sources of confusion.

ZSK and KSK describe ROLES. DNSKEY describes HOW their public keys are represented in DNS.

A signed zone commonly publishes multiple DNSKEY records. Some correspond to ZSKs; some correspond to KSKs.

ZSK: Zone Signing Key

The ZSK's job is to sign the zone's ordinary RRsets.



Mental model: the ZSK signs ordinary zone data and is usually easier to rotate than the KSK.

KSK: Key Signing Key

The KSK's primary job is to authenticate the zone's DNSKEY RRset.

Why separate the roles? The key used to establish trust with the parent can change less often, while the ZSK can rotate independently.

KSK does not mean "a different DNS record type." Its public half is also a DNSKEY record.

Simplified teaching model: ZSK signs zone data; KSK signs DNSKEY. Real deployments can use combined signing keys too.

How can two DNSKEY records look different?

DNSKEY

The DNSKEY flags field commonly distinguishes ZSK-like and KSK-like records.

```
example.com.        3600  DNSKEY  256 3 13 <ZSK-public-key>
```

```
example.com.        3600  DNSKEY  257 3 13 <KSK-public-key>
```

256 → Zone Key flag set (common ZSK presentation)

257 → Zone Key + SEP bit (common KSK presentation)

Understand the roles first; flags are merely how the records indicate intended use.

A signed zone now contains both data and security records

PUT TOGETHER

Let’s put the pieces together before introducing the parent.

```
www.example.com.      300   A      203.0.113.10
```

```
www.example.com.      300   RRSIG  A ... keytag=11111 ... <sig>
```

```
example.com.          3600  DNSKEY 256 ... <ZSK public>
```

```
example.com.          3600  DNSKEY 257 ... <KSK public>
```

```
example.com.          3600  RRSIG  DNSKEY ... keytag=22222 ... <sig>
```

ZSK public key verifies RRSIG(A)

KSK public key verifies RRSIG(DNSKEY)

Remaining problem: why should I trust the KSK?

The bootstrap problem

TRUST

If the zone tells you its own key, an attacker could forge both the data and the key.

Cryptography alone does not tell us which public key is legitimate.

We need an authenticated link from something we already trust.

DS: the parent’s pointer to the child’s key

DS

A DS record is stored in the parent zone and identifies a child zone’s KSK/DNSKEY.

Parent zone: com.

Child zone: example.com.

```
example.com.    86400 DS      22222 13 2 <digest-of-child-KSK>
```

```
example.com.    3600  DNSKEY  257 3 13 <KSK-public-key>
```

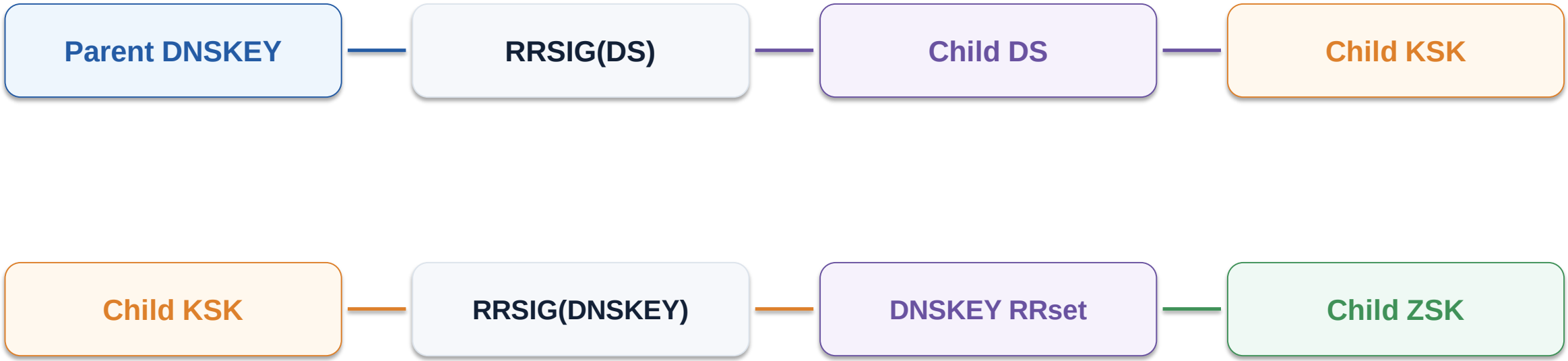
Validator hashes the child KSK/DNSKEY and checks whether it matches the DS digest published by the parent.

DS lives in the PARENT. DNSKEY lives in the CHILD.

One delegation step: parent → child

CHAIN

This is the core trust handoff in DNSSEC.

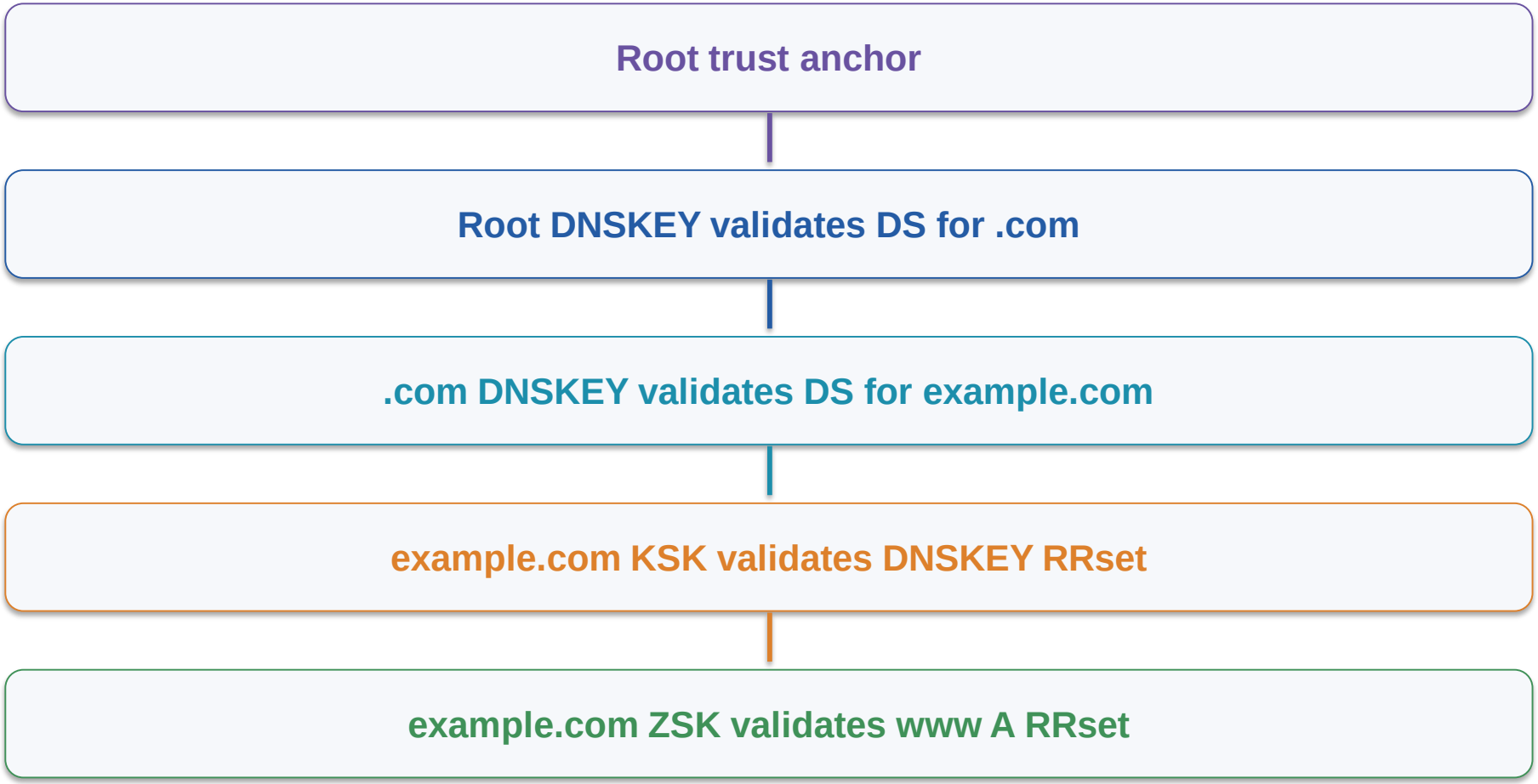


Trust crosses the zone boundary through DS, then continues inside the child using DNSKEY/RRSIG.

The complete chain of trust

CHAIN

A validating resolver begins with a root trust anchor and walks downward.



Every step is cryptographically checked. A broken step breaks validation below it.

Worked example: secure www.example.com

EXAMPLE

We will validate one A record slowly, one proof at a time.

Question	Answer	Need to prove
What is the IP of www.example.com?	203.0.113.10	This answer came from the legitimate example.com zone.

We will work backward from the answer until we reach something the resolver already trusts.

Step 1 — Receive the A RRset and its RRSIG

EXAMPLE

The signature says which DNSKEY should verify the A RRset.

```
www.example.com.      300   A      203.0.113.10
```

```
www.example.com.      300   RRSIG  A ... keytag=11111 example.com. <signature>
```

Need DNSKEY with key tag 11111

At this moment, we have a signature—but we do not yet know whether the key is trustworthy.

Step 2 — Fetch the example.com DNSKEY RRset

EXAMPLE

The matching ZSK public key is published as a DNSKEY record.

```
example.com.        3600  DNSKEY  256 3 13 <ZSK public>   keytag=11111
```

```
example.com.        3600  DNSKEY  257 3 13 <KSK public>   keytag=22222
```

```
example.com.        3600  RRSIG   DNSKEY ... keytag=22222 ... <signature>
```

ZSK verifies the A RRset

KSK verifies the DNSKEY RRset

Still need to establish that KSK 22222 is the legitimate child key.

Step 3 — Check the DS in the parent (.com)

EXAMPLE

The parent commits to the child KSK using a digest.

In .com zone

example.com. 86400 DS 22222 13 2 <digest>

In example.com zone

example.com. 3600 DNSKEY 257 3 13 <KSK public>

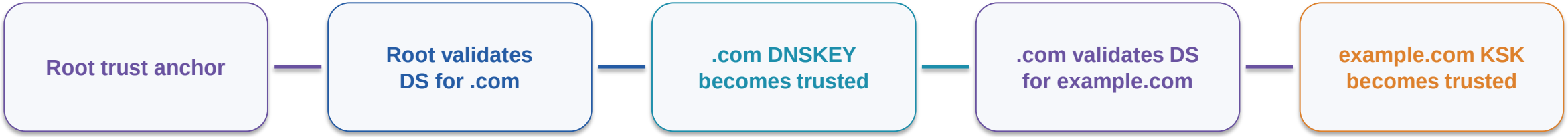
Compute digest(KSK DNSKEY) → compare with DS digest. Match = the parent is pointing to this child key.

But we must also validate the .com DS record itself.

Step 4 — Validate .com, then continue to the root

EXAMPLE

The same pattern repeats at every signed delegation.



Once the chain reaches the configured root trust anchor, the resolver has a cryptographic path from root → .com → example.com.

Now the ZSK can safely validate www.example.com A

Validation algorithm: a practical mental checklist

VALIDATION

For each signed zone, repeat the same small set of questions.

1. **Do I have the RRset + its RRSIG?**
2. **Which DNSKEY does the RRSIG identify?**
3. **Can that DNSKEY verify the signature?**
4. **Is the zone’s DNSKEY RRset itself authenticated?**
5. **Does the parent’s validated DS match the child key?**
6. **Can I continue upward to a trust anchor?**

Signature valid + key trusted = authenticated data

What result does a validating resolver produce?

VALIDATION

DNSSEC validation is not simply “signed” versus “unsigned”.

SECURE	INSECURE	BOGUS
A complete valid chain of trust exists and signatures verify.	The zone is intentionally unsigned; there is no DS establishing a signed child.	DNSSEC was expected, but validation failed: bad signature, wrong DS, expired RRSIG, etc.

Bogus commonly causes a validating resolver to return SERVFAIL.

What attacks does DNSSEC stop?

SECURITY

Assuming the resolver validates DNSSEC and the chain is correctly configured.

Protected against

- Forged DNS answers that do not carry a valid signature
- Tampering with signed RRsets in transit
- Substitution of an untrusted child key when a valid DS chain exists

Not solved by DNSSEC

- Traffic observation: queries are still visible
- DDoS against DNS infrastructure
- Compromise of authoritative signing keys
- Malicious data signed by the legitimate zone operator

DNSSEC authenticates DNS data; it does not make DNS confidential.

Bonus concept: how do you sign “this name does not exist”?

NSEC

A signature can authenticate existing data—but NXDOMAIN needs authenticated denial.

DNSSEC uses NSEC or NSEC3 records to provide cryptographically authenticated denial of existence.

For this lecture, keep the main chain-of-trust model separate: RRset → RRSIG → DNSKEY → DS → parent.

NSEC/NSEC3 is an extension of the same signed-data idea.

Why key rollovers matter

OPERATIONS

DNSSEC keys are operational credentials; they must sometimes be replaced without breaking trust.

Caches and TTLs mean key changes need overlap and careful timing.

ZSK rollover
Usually internal to the child zone

KSK rollover
May require coordinated DS update at parent

Operational mistakes can turn a valid signed zone into BOGUS.

Common misconceptions to eliminate

REVIEW

If students can answer these, the core model is solid.

“DNSKEY is another key type.”	No. DNSKEY is the DNS record carrying a public key.
“KSK signs all records.”	Teaching model: KSK signs the DNSKEY RRset; ZSK signs ordinary RRsets.
“RRSIG contains the public key.”	No. RRSIG contains a signature and metadata; public keys are in DNSKEY.
“DS is in the child zone.”	No. The parent publishes DS for the child.
“DNSSEC encrypts DNS.”	No. It authenticates data; confidentiality is separate.

Classroom exercise: identify each object

ACTIVITY

Give students 4–5 minutes, then discuss as a class.

A www.example.com. 300 A 203.0.113.10

B www.example.com. 300 RRSIG A ... 11111 example.com. <sig>

C example.com. 3600 DNSKEY 256 3 13 <key1>

D example.com. 3600 DNSKEY 257 3 13 <key2>

E example.com. 86400 DS 22222 13 2 <digest>

Questions: Which record is data? Which is a signature? Which public key is likely ZSK/KSK? Where should E actually be stored?

Exercise answer

ACTIVITY

The important part is explaining why, not just naming the records.

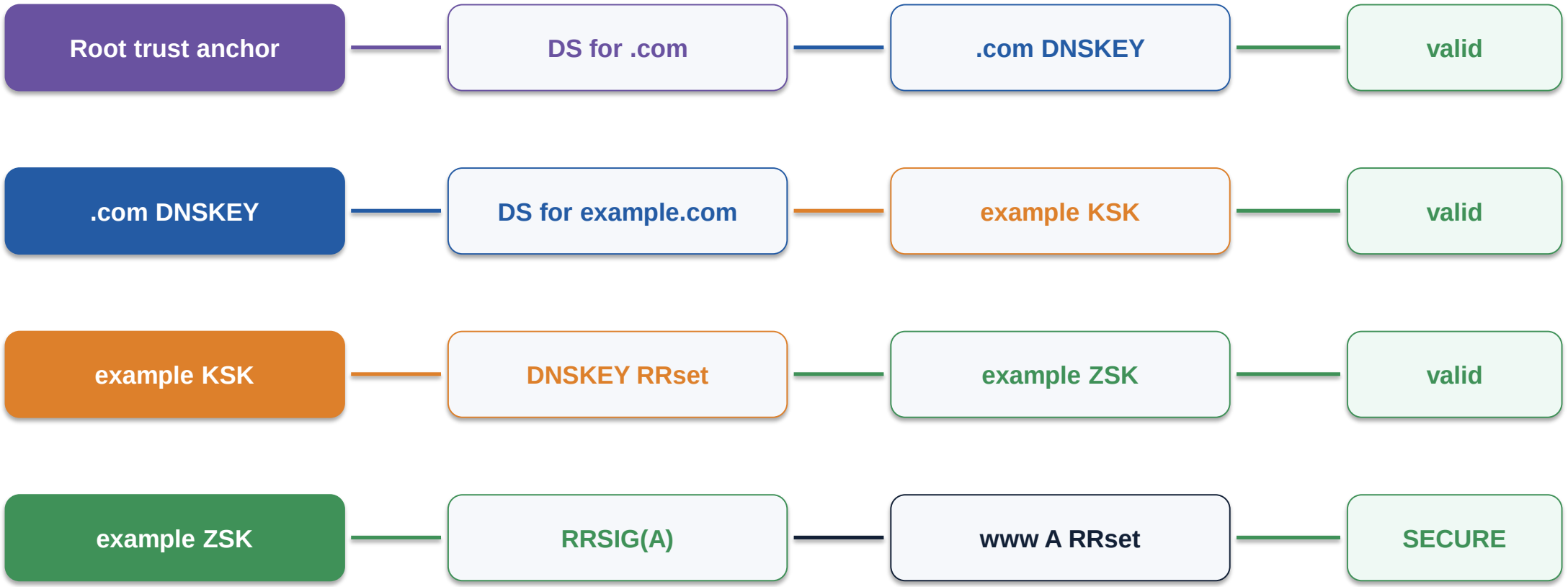
Reasoning

- A is the ordinary A RRset data.
- B is RRSIG(A); key tag 11111 points to the signing DNSKEY.
- C is a DNSKEY with flags 256: commonly the ZSK public key.
- D is a DNSKEY with flags 257: commonly the KSK public key.
- E is a DS record for example.com, but it belongs in the parent (.com) zone.
- Trust path: parent-validated DS → child KSK → child DNSKEY RRset → ZSK → A RRset.

The entire DNSSEC story on one slide

SUMMARY

Read from left to right: each proof makes the next object trustworthy.



Final recap: five sentences to remember

SUMMARY

If these are clear, the rest of DNSSEC becomes much easier.

1. A zone is an administrative portion of the DNS namespace with authoritative data.
2. **DNSSEC signs RRsets, and the resulting signature is stored in an RRSIG record.**
3. **ZSK and KSK are key roles; their public keys are published as DNSKEY records.**
4. The KSK authenticates the DNSKEY RRset; the ZSK authenticates ordinary zone RRsets.
5. **A parent's DS record links trust to the child, forming a chain from the root trust anchor to the final answer.**

DNSSEC = signed RRsets + trusted keys + parent-to-child delegation of trust.

Discussion / questions

END

Try to explain the chain without looking at the record syntax.

Can you explain why a valid RRSIG is not enough by itself?

Why does the DS record belong in the parent zone?

What is the difference between “KSK” and “DNSKEY”?

End goal: reason about trust, rather than memorize acronyms.